

РСА — Метод Главных Компонент

Зачем снижать размерность?

Реальные данные:

Задача	Число признаков
Изображение 64×64	4 096 пикселей
TF-IDF классификатор	100 000+ слов
Геномные данные	20 000+ генов

Проблемы высокой размерности:

- KNN, SVM перестают работать
- Обучение замедляется
- Переобучение

Проклятие размерности

В p -мерном пространстве:

$$\frac{d_{\min}}{d_{\max}} \xrightarrow{p \rightarrow \infty} 1$$

Все точки становятся **одинаково далёкими** друг от друга.

Идея: перейти из $\mathbb{R}^p$ в $\mathbb{R}^k$, $k \ll p$, сохранив **максимум информации**.

2D: $d_{\min}/d_{\max} \approx 0.30$ ← видим «близко» и «далеко»
10D: $d_{\min}/d_{\max} \approx 0.72$
100D: $d_{\min}/d_{\max} \approx 0.91$
1000D: $d_{\min}/d_{\max} \approx 0.97$ ← всё одинаково

Что такое «информация»?

Ответ: дисперсия (разброс) данных.

Направление с наибольшей дисперсией содержит наибольшую информацию.

Проекция точки $\mathbf{x}_i$ на направление $\mathbf{u}$, $\|\mathbf{u}\| = 1$:

$$z_i = \mathbf{x}_i^\top \mathbf{u} \in \mathbb{R}$$

Задача: найти $\mathbf{u}$, максимизирующий $\text{Var}(z_i)$.

Геометрия проекций

[Слайд с демонстрацией кода из ноутбука]

Три направления, три значения дисперсии:

- Угол 0° (ось X): $\text{Var} = 2.98$
- Угол 45° : $\text{Var} = 4.47$ ✓ лучше!
- Угол 120° : $\text{Var} = 0.83$

Есть ли оптимальное направление? Да — это собственный вектор ковариационной матрицы.

Ковариационная матрица

Центрируем данные: $X_c = X - \bar{X}$

$$\mathbf{C} = \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}$$

Элементы: $C_{ij} = \text{Cov}(x_i, x_j)$

Свойства:

- Симметрична: $C_{ij} = C_{ji}$
- Положительно полуопределена: $\lambda_i \geq 0$
- Диагональ C_{ii} — дисперсии признаков

```
X_c = X - X.mean(axis=0)
C = (X_c.T @ X_c) / (X.shape[0] - 1)
```

Дисперсия проекции через матрицу C

$$\text{Var}(\mathbf{u}) = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{x}_i^\top \mathbf{u})^2 = \mathbf{u}^\top \underbrace{\frac{X_c^\top X_c}{n-1}}_{\mathbf{C}} \mathbf{u}$$

Задача оптимизации:

$$\max_{\mathbf{u}} \mathbf{u}^\top \mathbf{C} \mathbf{u} \quad \text{при} \quad \|\mathbf{u}\| = 1$$

Метод Лагранжа:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{u}} [\mathbf{u}^\top \mathbf{C} \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1)] &= 0 \\ \Rightarrow \mathbf{C} \mathbf{u} &= \lambda \mathbf{u} \end{aligned}$$

Eigen Decomposition

По спектральной теореме (C симметрична):

$$\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$$

Обозначение	Смысл
$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_p)$	Собственные числа = дисперсия вдоль каждой оси
$\mathbf{V} = [\mathbf{v}_1 \mid \dots \mid \mathbf{v}_p]$	Собственные векторы = главные направления

Сортируем: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$

$\mathbf{v}_1$ — направление максимальной дисперсии → **первая главная компонента (PC1)**

Геометрический смысл

[Слайд с визуализацией из ноутбука — эллипс + стрелки PC1, PC2]

Данные формируют **эллипсоид** в $\mathbb{R}^p$.

Главные оси эллипсоида — это
собственные векторы C.

Длины полуосей пропорциональны $\sqrt{\lambda_i}$.

```
eigenvalues, eigenvectors = np.linalg.eigh(C)

# Sort descending
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

Алгоритм PCA

1. Центрирование: $X_c = X - \bar{X}$
2. Ковариация: $\mathbf{C} = \frac{1}{n-1} X_c^\top X_c$
3. Eigen decomp: $\mathbf{C} \mathbf{v}_i = \lambda_i \mathbf{v}_i$
4. Сортировка: $\lambda_1 \geq \lambda_2 \geq \dots$
5. Проекция: $W = [\mathbf{v}_1 | \dots | \mathbf{v}_k]$, $Z = X_c W$

Результат: $Z \in \mathbb{R}^{n \times k}$ — данные в пространстве главных компонент.
Признаки в Z некоррелированы: $Z^\top Z / (n - 1) = \Lambda_k$.

Explained Variance Ratio

Доля дисперсии, объяснённой i -й компонентой:

$$r_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

Как выбрать k ? — Накопленная дисперсия $\geq 90\%$:

$$\sum_{i=1}^k r_i \geq 0.90$$

```
pca = PCA()  
pca.fit(X)  
cumvar = np.cumsum(pca.explained_variance_ratio_)  
k = np.argmax(cumvar >= 0.90) + 1
```

PCA — реализация с нуля

```
class PCAFromScratch:
    def fit(self, X):
        self.mean_ = X.mean(axis=0)
        X_c = X - self.mean_
        C = (X_c.T @ X_c) / (X_c.shape[0] - 1)

        eigenvalues, eigenvectors = np.linalg.eigh(C)
        idx = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[idx]
        eigenvectors = eigenvectors[:, idx]

        self.components_ = eigenvectors[:, :self.n_components].T
        self.explained_variance_ratio_ = \
            eigenvalues[:self.n_components] / eigenvalues.sum()
        return self

    def transform(self, X):
        return (X - self.mean_) @ self.components_.T
```

Свойство: декорреляция

До PCA:

$\text{Corr}(x_1, x_2) = 0.913$ ← сильная корреляция

После PCA:

$\text{Corr}(PC1, PC2) = 0.000$ ← полная декорреляция

PCA поворачивает систему координат так, чтобы оси совпали с главными направлениями дисперсии. В новом базисе признаки линейно независимы.

SVD — тот же результат, лучше численно

Сингулярное разложение центрированных данных:

$$X_c = U\Sigma V^\top$$

Связь с PCA:

$$X_c^\top X_c = V\Sigma^2 V^\top \Rightarrow \lambda_i = \frac{\sigma_i^2}{n-1}$$

Главные компоненты = правые сингулярные векторы V .

Проекция без вычисления C :

$$Z = U_k \Sigma_k = X_c V_k$$

SVD vs Eigen decomposition

	<code>np.linalg.eigh(C)</code>	<code>np.linalg.svd(X_c)</code>
Входные данные	$C = X_c^\top X_c$	X_c напрямую
Числовая стабильность	✗ квадрирование σ	✓ без потери точности
Вычислительная сложность	$O(p^3)$	$O(np^2)$ при $n > p$
Используется в sklearn	—	✓ PCA внутри

Вывод: В production — всегда SVD (через sklearn). `eigh` — для понимания математики.

Low-rank Approximation

Теорема Эккарта-Янга (1936):

$$A_k = U_k \Sigma_k V_k^\top = \arg \min_{\text{rank}(B)=k} \|A - B\|_F$$

Ошибка приближения:

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

РСА — **оптимальное** линейное снижение размерности по MSE реконструкции.

Сжатие изображений

[Слайд с демонстрацией из ноутбука]

k	Дисперсия	Сжатие
1	45%	100×
10	82%	15×
50	97%	3.5×
200	99.9%	1.1×

Размер k -ранговой аппроксимации: $k(n + p + 1)$ чисел vs np оригинал.

Применения low-rank

Область	Матрица	k-rank даёт
Рекомендации	Пользователь × фильм	Скрытые факторы вкусов
NLP (LSA)	Документ × слово	Скрытые темы
Шумоподавление	Сигнал + шум	Первые k = сигнал
LoRA (LLM)	Веса нейросети	Эффективное дообучение
Компрессия изображений	Пиксели	JPEG использует DCT $\approx$ SVD

PCA на реальных данных: Digits

[Слайд с 2D-визуализацией из ноутбука]

MNIST Digits: 1797 изображений, 64 признака

- Для 90% дисперсии достаточно **~21 компоненты** из 64
- 2D-проекция показывает чёткие кластеры цифр
- PC1 и PC2 вместе объясняют лишь ~25% дисперсии

2D PCA ≠ полная картина. Используйте k компонент для ML, 2D — только для визуализации.

Eigendigits

[Слайд с первыми 16 главными компонентами как изображениями]

Каждая главная компонента — **линейная комбинация всех пикселей**.

PC1 $\approx$ «средняя цифра»

PC2 $\approx$ «вертикальность vs горизонтальность»

PC3 $\approx$ «округлость»

...

Аналогия: **eigenfaces** — классический метод распознавания лиц (Turk & Pentland, 1991).

Чек-лист: как применять PCA

1.  Обработать пропуски
2.  **Стандартизировать** (`StandardScaler`) — обязательно!
3.  Выбрать k по `explained_variance_ratio_`
4.  Обучить `scaler` и `pca` только на train!
5.  Сохранить `scaler` + `pca` вместе с моделью

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.90)), # 90% variance
    ('clf', LogisticRegression()),
])
pipe.fit(X_train, y_train)
```

Ограничения PCA

Ограничение	Следствие	Решение
Только линейные зависимости	Нелинейные структуры теряются	Kernel PCA, t-SNE, UMAP
Чувствительность к масштабу	Большие числа доминируют	<code>StandardScaler</code> перед PCA
Нет меток классов	Макс. дисперсия $\neq$ макс. разделимость	LDA (supervised)
Нет интерпретируемости	Компоненты $\neq$ исходные признаки	Sparse PCA, Feature Selection

Когда PCA не работает

[Слайд с визуализацией из ноутбука: кольца, Swiss roll]

Два кольца (2D → 1D)

Линейная проекция смешивает классы.
Нужна нелинейная карта.

Swiss roll (3D → 2D)

PCA «раздавливает» спираль. t-SNE/UMAP сохраняет топологию.

Если данные лежат на **нелинейном многообразии** — PCA не поможет.

Альтернативы PCA

Метод	Когда	Особенность
Kernel PCA	Нелинейность, малый n	Медленнее, нет <code>inverse_transform</code>
t-SNE	Визуализация 2D/3D	Только для viz, не для ML
UMAP	Визуализация + ML	Быстрее t-SNE, сохраняет глобальную структуру
ICA	Источники сигналов (BSS)	Независимость (не корреляция)
NMF	Тексты, изображения, $X \geq 0$	Интерпретируемые части
Autoencoder	Нелинейность, много данных	Требует обучения нейросети

Итог: PCA в трёх строках

```
X_c = X - X.mean(axis=0) # 1. Center
U, S, Vt = np.linalg.svd(X_c, full_matrices=False) # 2. SVD
Z = U[:, :k] * S[:k] # 3. Project
```

Что сохраняется: направления максимального разброса

Что теряется: $(1 - \sum_{i=1}^k r_i)$ дисперсии

Оптимальность: по норме Фробениуса (теорема Эккарта-Янга)

Резюме

Применяем PCA когда:

- p велико (признаки $\gg$ объекты)
- Мультиколлинеарность
- Нужна 2D/3D визуализация
- Шумоподавление

Не применяем когда:

- Нелинейные зависимости
- Нужна интерпретируемость признаков
- Мало признаков ($< 10-20$)
- Уже есть domain knowledge о важных признаках

Всегда стандартизировать данные перед PCA!

ДЗ 6 (часть 1)

1. Загрузить датасет с ≥ 100 признаками
2. Стандартизировать, построить elbow-plot explained variance
3. Выбрать k для 90% дисперсии
4. Обучить LogReg в исходном и PCA-пространстве
5. Сравнить ассурасу и время обучения
6. Нарисовать 2D PCA-проекцию с метками классов

Вопрос для защиты: почему нужно центрировать данные? Что произойдёт, если не центрировать?

Полезные ссылки

- [StatQuest: PCA Step-by-Step \(YouTube\)](#)
- [sklearn.decomposition.PCA](#)
- [Lilian Weng: From Auto-Encoder to Beta-VAE](#)
- Bishop "PRML", Chapter 12: Continuous Latent Variables